# Experiment 8

**Student Name: DIVYANSH** **UID: 23BCS11778**
**Branch: CSE** **Section/Group: KRG 2A**
**Semester: 6th** **Date of Performance: 08/04/2026**
**Subject Name: System Design** **Subject Code: 23CSH-314**

1. **Aim**: To design and analyze a scalable **Stock Trading Platform** (like Zerodha/Groww/Upstox) that allows users to trade stocks in real-time, track market prices, manage portfolios, and perform transactions efficiently using distributed system architecture.

2. **Objective**:

- To identify functional and non-functional requirements of a stock trading system
- To design High Level Architecture (HLD) for trading platforms
- To understand real-time stock price streaming
- To design microservices for trading, portfolio, and payments
- To study order execution and validation mechanisms
- To understand scalability challenges in financial systems

3. **Tools:**

- Draw.io / Excalidraw – System design diagrams
- MySQL / PostgreSQL – User & transaction data
- Redis – Caching & real-time data
- Kafka – Event streaming (stock prices/orders)
- Load Balancer – Traffic distribution
- WebSockets – Real-time stock updates
- Microservices Architecture
- InfluxDB – Time-series stock data

4. **System Requirements:**

### A. Functional Requirements:-

- Users should be able to register, login, and complete KYC
- Users should view real-time stock prices and historical data
- Users should place buy/sell orders
- Users should modify or cancel orders
- System should provide a watchlist with live updates

- Users should view portfolio (holdings, P&L, history)
- Users should deposit/withdraw funds
- System should validate orders before execution
- System should show order status in real-time

### B. Non-Functional Requirements:-

**1) Scalability**

a. Support 2–3 million daily active users

b. Handle 8k–10k stocks

**2) Consistency (CAP Theorem)**

a. Consistency > Availability

b. No duplicate trades / wrong balances

**3) Availability**

a. Real-time stock prices should always be available

**4) Latency**

a. Order placement < 100 ms

b. Market data < 50 ms

**5) Reliability**

a. No loss of trade or order data

## 5. High Level Design (HLD):

System follows Microservices Architecture

**Main Components:**

**1) API Gateway**

a. Authentication

b. Authorization

c. Routing

d. Rate Limiting

2) **User Service**
    a. Registration & login
    b. KYC verification
    c. User DB (MySQL)
3) **Price Tracker Service**
    a. Fetches real-time stock data from exchange (NSE/BSE)
    b. Uses Kafka + Redis for streaming
4) **Order Service**
    a. Handles buy/sell orders
    b. Stores order data
5) **Validation Service**
    a. Checks KYC
    b. Checks funds availability
    c. Prevents invalid trades
6) **Portfolio Service**
    a. Tracks holdings
    b. Calculates P&L
7) **Watchlist Service**
    a. Stores user watchlist
    b. Provides real-time updates
8) **Payment Service**
    a. Deposit / Withdraw funds
9) **Exchange Server**
    a. External system (NSE/BSE) for trade execution

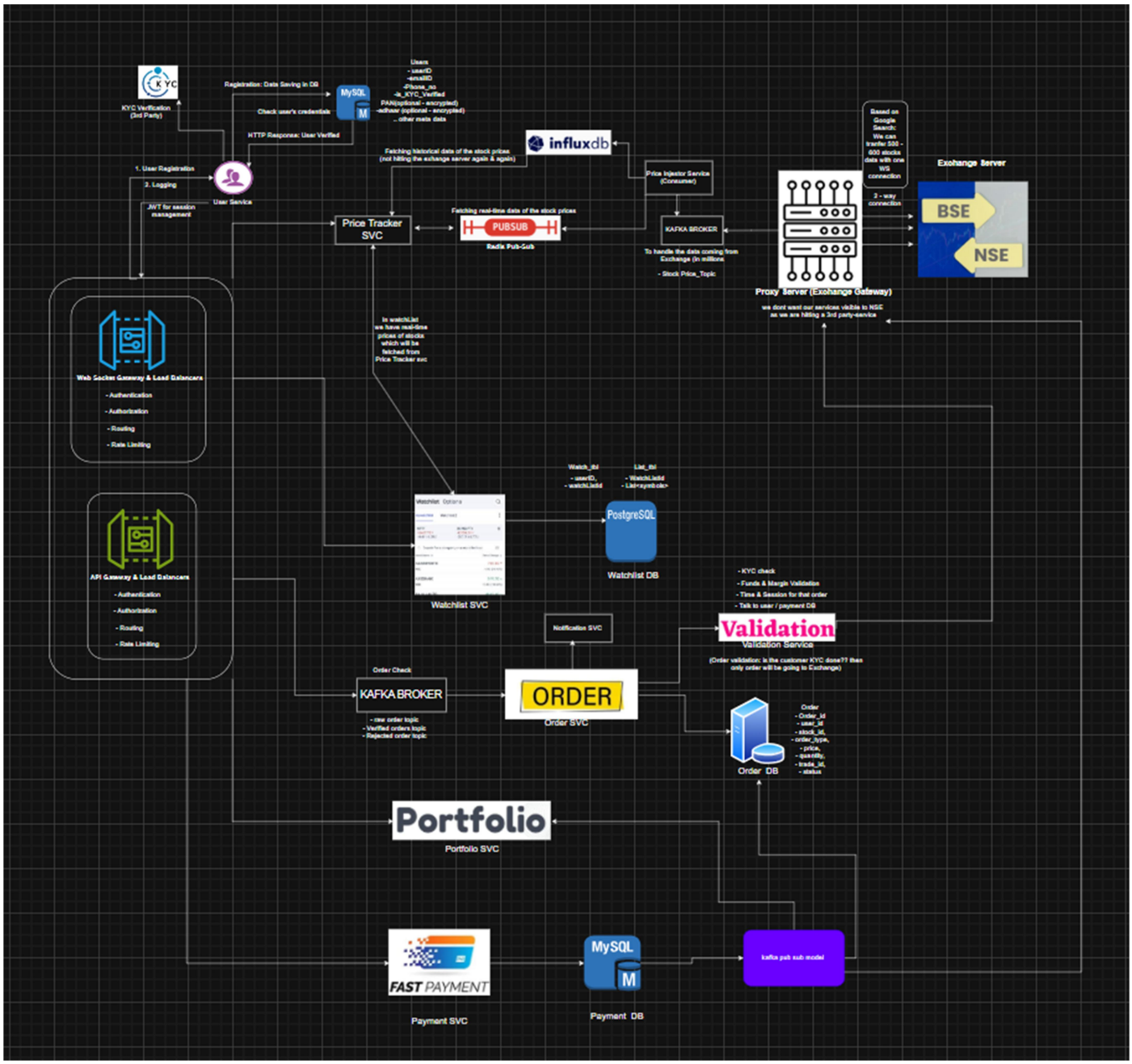
## 6. Trading Flow (Step-by-Step):

1) User logs in and completes KYC
2) User views stock prices (real-time via WebSockets)
3) User adds stocks to watchlist
4) User places buy/sell order
5) Order goes to Validation Service
6) If valid → sent to Exchange Server
7) Order executed
8) Order status updated
9) Portfolio updated
10) User sees updated P&L

## 7. Scalability Solution:

To support millions of users:

- Use Kafka for real-time stock streaming
- Use Redis for caching stock prices
- Use WebSockets for live updates
- Use Load Balancers for traffic distribution
- Use Microservices architecture
- Use separate databases (User, Order, Portfolio, Payment)
- Use InfluxDB for time-series stock data
- Use API Gateway for centralized control


KYC Verification (3rd Party)
Registration: Data Saving in DB
MySQL
Check user's credentials
HTTP Response: User Verified
User
- userID
-emailID
-Phone_no
-is_KYC_Verified
PAN(optional - encrypted)
-adhaar (optional - encrypted)
.. other meta data
influxdb
Fetching historical data of the stock prices (not hitting the exhange server again & again)
1. User Registration
2. Logging
User Service
JWT for session management
Price Tracker SVC
Fetching real-time data of the stock prices
PUBSUB
Redis Pub-Sub
Price Injector Service (Consumer)
KAFKA BROKER
To handle the data coming from Exchange (in millions)
- Stock Price_Topic
Based on Google Search: We can transfer 500 - 600 stocks data with one WS connection
2 - way connection
Exchange Server
BSE
NSE
Proxy Server (Exchange Gateway)
we dont want our services visible to NSE as we are hitting a 3rd party-service
Web Socket Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
In watchList we have real-time prices of stocks which will be fetched from Price Tracker svc
API Gateway & Load Balancers
- Authentication
- Authorization
- Routing
- Rate Limiting
Watch_tbl
- userID,
- watchListId
List_tbl
- WatchListId
- List<symbol>
PostgreSQL
Watchlist DB
Watchlist SVC
- KYC check
- Funds & Margin Validation
- Time & Session for that order
- Talk to user / payment DB
Validation
Validation service
(Order validation: is the customer KYC done?? then only order will be going to Exchange)
Notification SVC
Order Check
KAFKA BROKER
- raw order topic
- Verified orders.topic
- Rejected order topic
ORDER
Order SVC
Order DB
Order
- Order_id
- user_id
- stock_id,
- order_type,
- price,
- quantity,
- trade_id,
- status
Portfolio
Portfolio SVC
FAST PAYMENT
Payment SVC
MySQL
Payment DB
kafka pub sub model

## 8. Learning Outcomes (What I Have Learnt):

- Learned how stock trading platforms are designed
- Understood real-time data streaming using Kafka
- Learned order validation and execution flow
- Understood portfolio management systems
- Learned importance of consistency in financial systems
- Understood microservices and scalability
- Learned how exchanges (NSE/BSE) integrate with apps